

CURSO 2: ALGORITMOS AVANZADOS Y ESTRUCTURAS DE DATOS

CLASE 01 • NIVEL 2 - INTERMEDIO

Clase 01: Análisis de Complejidad y Notación Big-O

«Medir el Rendimiento de un Algoritmo a Medida que Crece la Entrada»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 01** del **Curso 2: Algoritmos Avanzados y Estructuras de Datos**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

Objetivos de la Sesión

Competencia Conceptual: Comprender la notación asintótica Big-O, análisis temporal y espacial en el peor caso.

Competencia Práctica: Identificar y clasificar complejidades $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$ y $O(n^2)$.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 01: Análisis de Complejidad y Notación Big-O

La notación Big-O describe cómo escala el tiempo de ejecución y el uso de memoria de un algoritmo.

☀ Metáfora Central: Medir el Rendimiento de un Algoritmo a Medida que Crece la Entrada

Big-O es como calcular cuánta gasolina consumirá un camión de carga según el número de kilómetros y peso.

Principios Teóricos y Modelo Mental

Nos enfocamos en el peor caso (Worst-case scenario) y descartamos constantes y términos de menor orden.

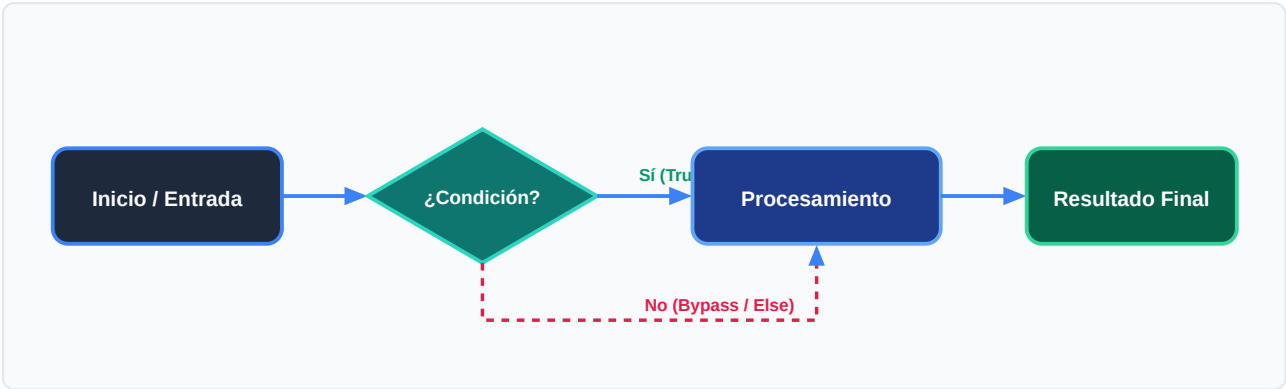
Un algoritmo $O(n)$ sobre 1 millón de elementos tarda milisegundos; un $O(n^2)$ puede tardar horas.

⚡ Regla de Oro en Python

Evita los bucles anidados innecesarios para prevenir la degradación a $O(n^2)$.

Arquitectura y Movimiento de Datos en Memoria

Comparativa de curvas de crecimiento asintótico frente a volumen de datos n .



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Cálculo de operaciones elementales.	Función matemática $T(n)$.
2. Evaluación	Eliminación de coeficientes constantes.	Simplificación algebraica.
3. Transformación	Identificación del término dominante.	Clase de complejidad Big-O.
4. Retorno / Salida	Benchmarking experimental con <code>time.perf_counter()</code> .	Validación empírica en ms.

Visualización Mental

Un bucle simple es $O(n)$; dos bucles anidados son $O(n^2)$; dividir a la mitad es $O(\log n)$.

Código de Demostración en Python 3.10+

Comparación práctica de tiempo $O(1)$ vs $O(n)$:

main.py (Python 3.10+)

PEP 8 Compliant

```
import time

def acceso_o1(lista: list, idx: int):
    return lista[idx] #  $O(1)$ 

def busqueda_on(lista: list, target: int):
    for item in lista: #  $O(n)$ 
        if item == target:
            return True
    return False

datos = list(range(1_000_000))
print("O(1) Acceso:", acceso_o1(datos, 500_000))
print("O(n) Búsqueda:", busqueda_on(datos, 999_999))
```

Análisis de Ingeniería del Código

El acceso por índice es inmediato $O(1)$, mientras que recorrer secuencialmente es lineal $O(n)$.



Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Trampas comunes al evaluar complejidad en Python.

⚠ Gotcha Frecuente (Trampa de Principiante)

Usar 'if x in lista:' dentro de un bucle for convierte tu código silenciosamente en $O(n^2)$.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
for elem in lista_a:
    if elem in lista_b: # ❌ 'in' en lista es
                        # O(n), total O(n^2)
        comunes.append(elem)
```

✅ Patrón Correcto

```
set_b = set(lista_b) # O(n)
for elem in lista_a:
    if elem in set_b: # ✅ 'in' en set es
                    # O(1), total O(n)
        comunes.append(elem)
```

🛡 Consejo de Resiliencia en Producción

Convierte listas a sets antes de hacer múltiples búsquedas de pertenencia.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 01: Análisis de Complejidad y Notación Big-O**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Medir el Rendimiento de un Algoritmo a Medida que Crece la Entrada
2. Regla de Oro	Evita los bucles anidados innecesarios para prevenir la degradación a $O(n^2)$.
3. Gotcha Crítico	Usar 'if x in lista:' dentro de un bucle for convierte tu código silenciosamente en $O(n^2)$.
4. Buenas Prácticas	Convierte listas a sets antes de hacer múltiples búsquedas de pertenencia.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Escribe un script que compare el tiempo real de buscar un elemento en una lista vs un set de 500.000 elementos.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.